

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Thu 18 JANUARY 2018 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

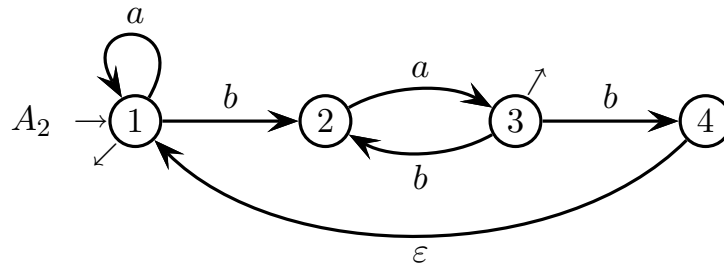
- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R_1 below, over the two-letter alphabet $\{a, b\}$:

$$R_1 = (a \mid \varepsilon)^+ (ba \mid bab)^*$$

Consider also the non-deterministic automaton A_2 below, over the same two-letter alphabet $\{a, b\}$, with a spontaneous transition, and with final nodes 1 and 3:



Answer the following questions (use the spaces / tables on the next pages):

- (a) Determine whether expression R_1 is ambiguous, and explain your answer.
- (b) By first using the Berry-Sethi method and then minimizing if necessary, find a deterministic and minimal automaton A_1 equivalent to expression R_1 .
- (c) Determine whether the language $L(R_1)$ generated by expression R_1 is local, and explain your answer.
- (d) Find an automaton, not necessarily deterministic, equivalent to automaton A_2 , but without spontaneous transitions.
- (e) (optional) What is the relationship between the language generated by expression R_1 and the language accepted by automaton A_2 ? Explain your answer.

question (a)

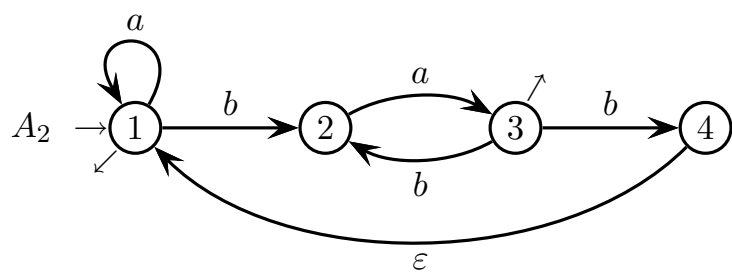
please build here the Berry-Sethi automaton A_1 of expression R_1 – question (b)

$$R_{\#,1} =$$

initials	
terminals	followers – the number of rows is not significant

draw here the state-transition graph of automaton A_1

please construct here the automaton without spontaneous transitions – question (d)
state-transition graph of the original automaton A_2



state-transition graph of the automaton without spontaneous transitions

rest of the exercise – questions (c), (e), and (a) if necessary

2 Free Grammars and Pushdown Automata 20%

1. Consider the following language L over the two-letter alphabet $\{a, b\}$:

$$L = \{ a^m b^n \mid m \geq n \geq 0 \ \wedge \ m - n \text{ is even} \}$$

Sample valid strings:

$$\varepsilon, a^2, ab, a^3b \in L$$

Sample invalid strings:

$$b, a^2b \notin L$$

Answer the following questions:

- (a) Write an *ambiguous BNF* grammar G_1 that generates language L .
 - (b) Draw at least two syntax trees for string $aaaa$ according to grammar G_1 .
 - (c) Write an *unambiguous BNF* grammar G_2 that generates language L .
 - (d) Draw the (unique) syntax tree for string $aaaa$ according to grammar G_2 .
-

2. Consider the fragment of a programming language that consists of assignments of two types of arithmetic expression, one simple and one complex, to variables. This fragmentary language is informally specified as follows:

- Each phrase of the language is a non-empty list of assignments separated by semicolon “;”.
- An assignment consists of a variable name, an assignment operator and an arithmetic expression.
- There are two assignment operators: “=” and “:=”. The former wants a *simple* expression on the right, and the latter wants a *complex* one instead.
- A *simple* arithmetic expression has at most two levels, without parentheses. It can use addition “+” and multiplication “*”, of variables and constants. The associativity of addition and multiplication is unspecified. Precedence is uncommon: multiplication has lower priority and addition higher.
- A *complex* arithmetic expression has arbitrarily many levels, with parenthesized subexpressions. It can use addition “+” and multiplication “*”, of variables and constants. Addition is left-associative and multiplication is right-associative. Precedence is as usual: multiplication has higher priority and addition lower.
- A variable identifier is alphanumerical (for simplicity use only lowercase letters), must have at least one heading letter and may freely contain underscores “_”.
- A constant is a decimal integer and may have a unary minus sign “-”.

For any unspecified minor detail, see the example below, and you may also follow the usual conventions, e.g., those of the C language. Example:

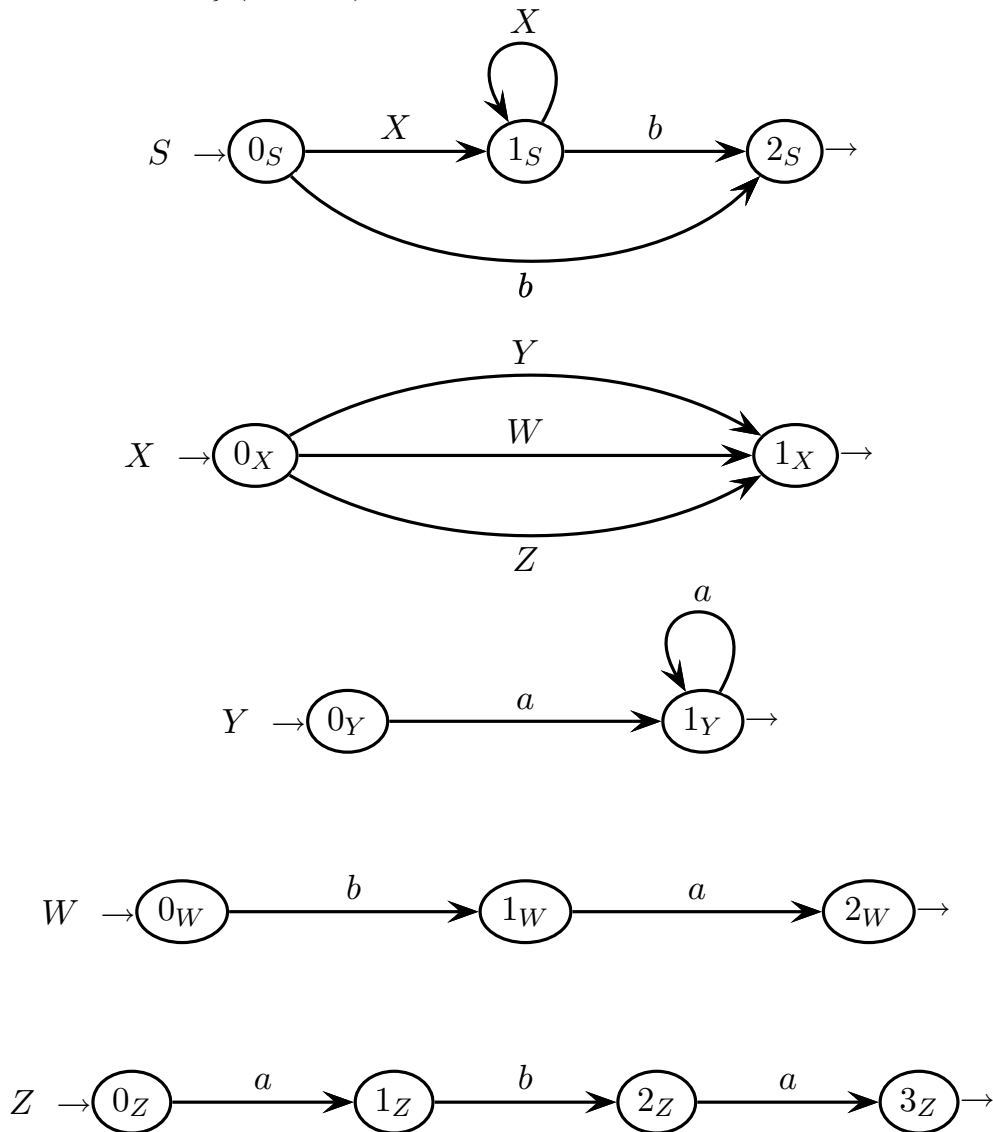
```
a = b + c + 2 * d;  
  
e := f + g + h * 3;  
  
a1 = a + -5 * c * d + b;  
  
b34_c := a * (c + d) * -040;
```

Answer the following questions:

- (a) Write a non-ambiguous grammar, in general of type *EBNF*, that models the described fragmentary language.
- (b) (optional) In order to (partially) test the behaviour of your grammar, separately draw the two syntax subtrees of the first two sample assignments. You may condense the trees by schematizing variables and constants with a terminal “a”.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ and the five-letter nonterminal alphabet $V = \{ S, X, Y, W, Z \}$ (axiom S):

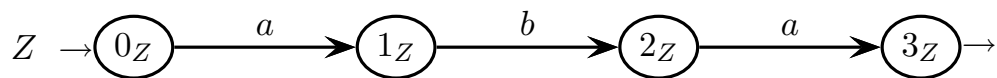
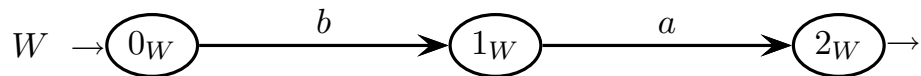
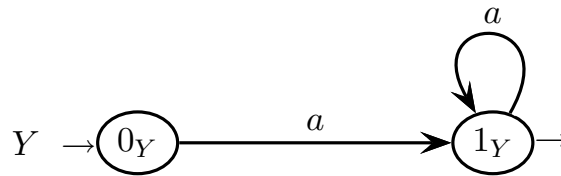
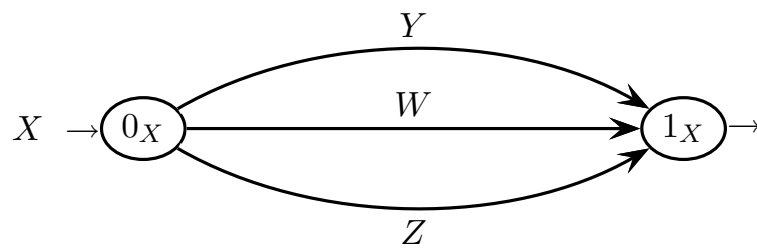
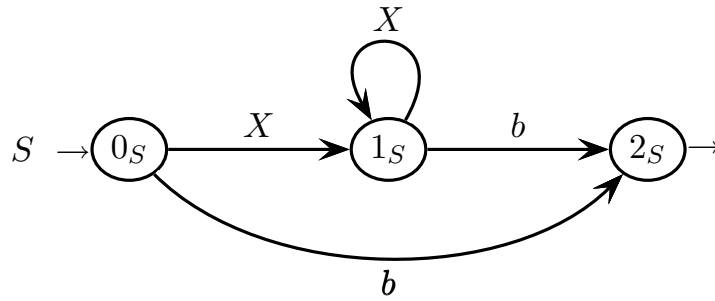


Answer the following questions (use the spaces / figures / tables on the next pages):

- (a) Draw a portion of the pilot graph sufficient to show that grammar G is not of type $ELR(1)$. Explain why grammar G is not of type $ELR(1)$.
- (b) Draw all the guide sets and, by using them, show that grammar G is not $ELL(1)$.
- (c) By using the Earley method, analyze the string $abab$ according to grammar G and show, based on the results of the analysis, that the string is accepted.
- (d) (optional) Argue, as simply as possible, that the language $L(G)$ admits an $ELR(1)$ grammar and even an $ELL(1)$ grammar.

please draw here the (sufficient fragment of the) pilot graph – question (a)

please draw here all the call arcs and write all the guide sets – question (b)



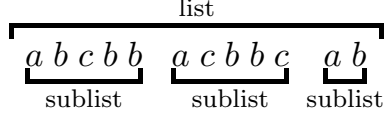
Early vector of string *a b a b* (to be filled in) – question (c)
(the number of rows is not significant)

[illegible]

question (d)

4 Language Translation and Semantic Analysis 20%

1. Consider a source language L_s of two-level nested lists. The alphabet is $\Sigma = \{ a, b, c \}$. A letter a heads each sublist, and the letters b and c are the elements of the sublists. A sublist may be empty, as well as the whole list. Example of a nested list:



A source grammar G_s that generates such a language L_s is the following (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow a X S \\ S \rightarrow \varepsilon \\ X \rightarrow b X \\ X \rightarrow c X \\ X \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions (use the schemes / spaces on the next pages):

- (a) Consider the translation τ_1 that maps each source list of L_s to a translated one as follows: the letter a of each sublist is moved to the trailing position of the sublist, and inside each sublist all the letters b precede all the letters c . Example:

$$\tau_1 (a b c b b \ a c b b c \ a b) = b b b c a \ b b c c a \ b a$$

Write a translation scheme (or grammar) with source grammar G_s that computes translation τ_1 . Do not change the structure of the source grammar G_s .

- (b) Consider the translation τ_2 that maps each source list of L_s to a translated one as follows: first in each sublist the numbers of letters b and c are exchanged, and then from left to right the sublist includes all the letters b , the letter a and all the letters c . Example:

$$\tau_2 (a b c b b \ a c b b c \ a b) = b a c c c \ b b a c c \ a c$$

Write a translation scheme (or grammar) with source grammar G_s that computes translation τ_2 . Do not change the structure of the source grammar G_s .

- (c) Are translations τ_1 and τ_2 invertible ? Please explain your answer.
- (d) (optional) Consider also the destination language of translation τ_1 , i.e., the language $L_{d_1} = \tau_1 (L_s)$, and assume that translation τ_2 has language L_{d_1} as its source language, instead of language L_s . Example:

$$\tau_2 (b b b c a \ b b c c a \ b a) = b a c c c \ b b a c c \ a c$$

Is translation τ_2 , modified as said above, invertible? Please explain your answer.

write on the right the dest. gram. or the whole transl. gram. for τ_1 – question (a)

$$\left\{ \begin{array}{l} S \rightarrow a X S \\ S \rightarrow \varepsilon \\ X \rightarrow b X \\ X \rightarrow c X \\ X \rightarrow \varepsilon \end{array} \right.$$

write on the right the dest. gram. or the whole transl. gram. for τ_2 – question (b)

$$\left\{ \begin{array}{l} S \rightarrow a X S \\ S \rightarrow \varepsilon \\ X \rightarrow b X \\ X \rightarrow c X \\ X \rightarrow \varepsilon \end{array} \right.$$

rest of the exercise – questions (c) and (d)

2. Consider the following abstract syntax (axiom S), over the two-letter alphabet $\{ a, b \}$:

$$\begin{cases} 1: S \rightarrow a S b S \\ 2: S \rightarrow a S \\ 3: S \rightarrow S b \\ 4: S \rightarrow \varepsilon \end{cases}$$

A sample valid string is: $a^3 b^2$ (see also the syntax tree on the next pages).

Assume that a syntax tree is given for each string. We want to compute, in the root node of any tree, the following values:

- the depth (distance from tree root) of the *deepest* leaf node labeled a
- the depth (distance from tree root) of the *least deep* leaf node labeled b

It is assumed that, if the tree does not include any leaf node of a given kind, then the required depth value is conventionally set equal to 0.

Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
da	left	integer	S	distance, from the current node S , of the <i>deepest</i> leaf node of type a in the subtree rooted at the current node conventionally set equal to 0 if the subtree does not include any leaf node of type a
db	left	integer	S	distance, from the current node S , of the <i>least deep</i> leaf node of type b in the subtree rooted at the current node conventionally set equal to 0 if the subtree does not include any leaf node of type b

Answer the following questions (use the tables / trees / spaces on the next pages):

- Write an attribute grammar G_a , based on the above syntax and using only the attribute da , that computes in the tree root the value of the depth of the *deepest* leaf node labeled a , or 0 if the tree does not include any node labeled a .
- Decorate the tree of string $a^3 b^2$, prepared on the next page, with the values of attribute da at every non-leaf node.
- Write another attribute grammar G_b , so as to compute in the tree root the value of the depth of the *least deep* leaf node labeled b , or 0 if the tree does not include any node labeled b , based on the above syntax and using only the attribute db .
- Decorate the tree of string $a^3 b^2$, prepared on the next page, with the values of attribute db at every non-leaf node.

for exercise 4.2 see the next pages
here space for continuation if necessary – all exercises

#	<i>syntax</i>	<i>semantics of G_a – question (a)</i>
---	---------------	---

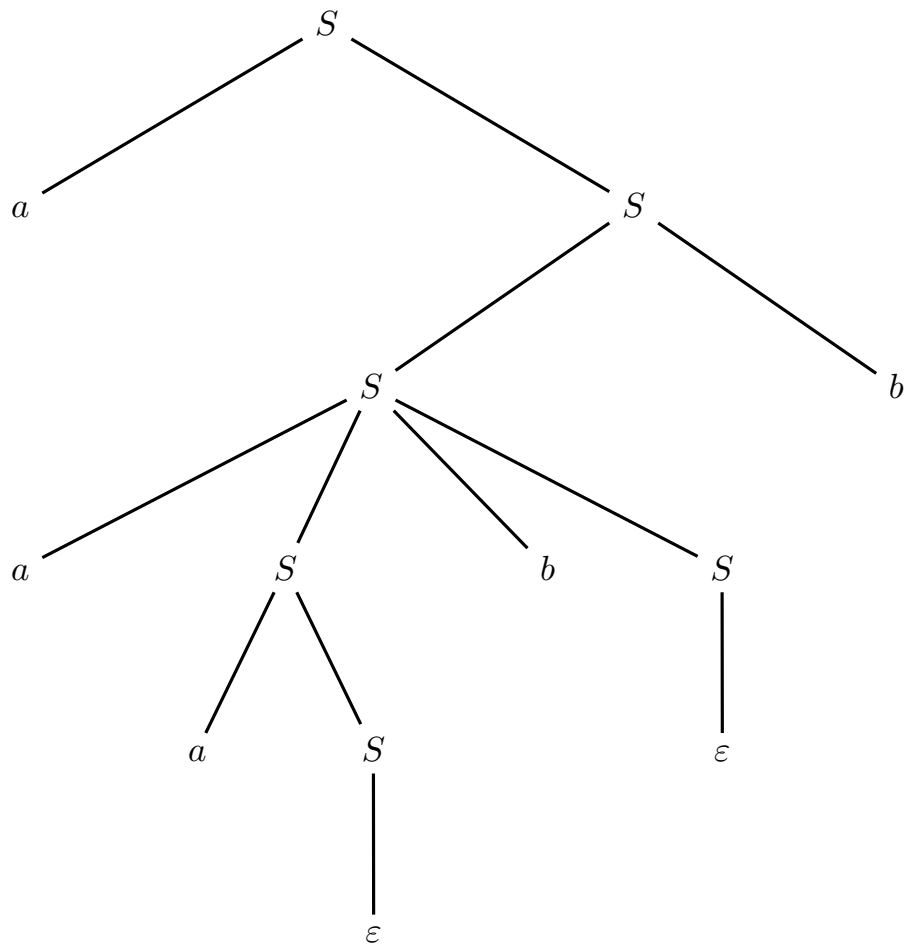
1:	$S_0 \rightarrow a S_1 b S_2$	
----	-------------------------------	--

2:	$S_0 \rightarrow a S_1$	
----	-------------------------	--

3:	$S_0 \rightarrow S_1 b$	
----	-------------------------	--

4:	$S_0 \rightarrow \varepsilon$	
----	-------------------------------	--

please decorate this tree with attribute da – question (b)



#	<i>syntax</i>	<i>semantics of G_b – question (c)</i>
---	---------------	---

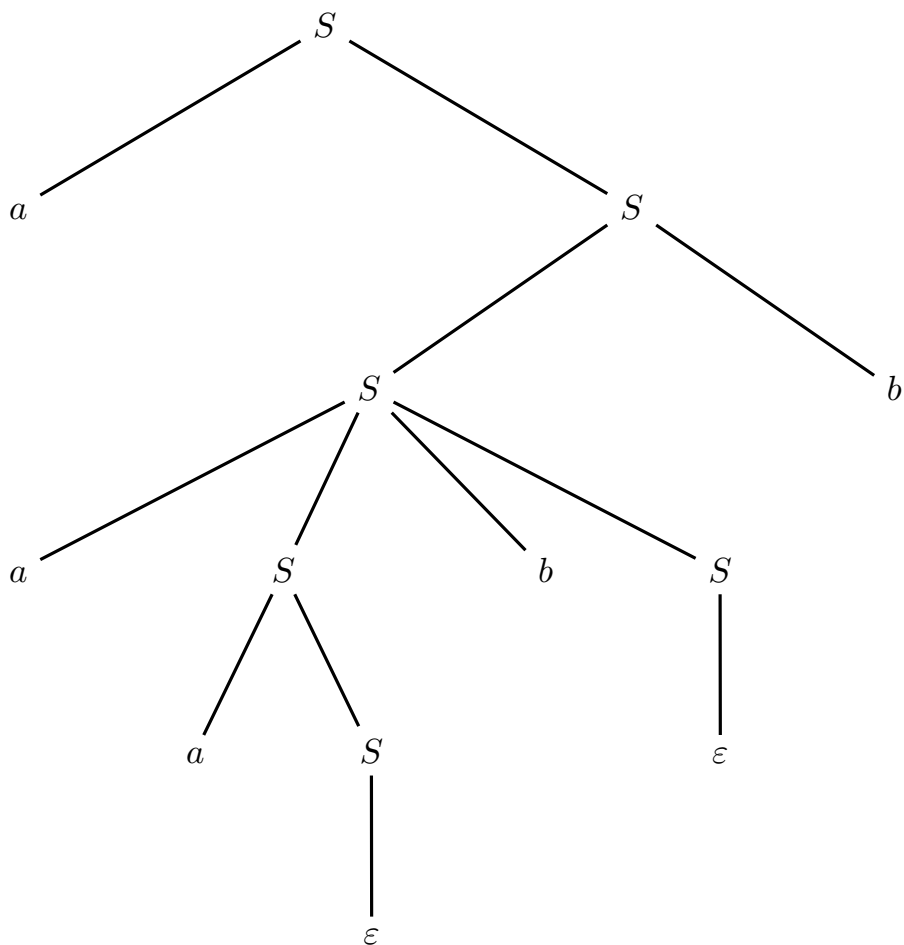
1: $S_0 \rightarrow a S_1 b S_2$

2: $S_0 \rightarrow a S_1$

3: $S_0 \rightarrow S_1 b$

4: $S_0 \rightarrow \varepsilon$

please decorate this tree with attribute db – question (d)



space for continuation if necessary – all exercises